Department of Computer Science

Machine Learning
COMP4388

Project 2

Amal Zaben 1211482
Hanadi Za’reer 1222729

Instructor: Dr. Radi Jarrar

Contents

1. Introduction: 3
2. Dataset Description: 3
3. Methodology and Interpretation: 4
   3.1 Overall Workflow 4
   3.2 Data Loading and Preprocessing 4
   3.3 Baseline CNN Model 5
   3.4 Improved CNN Model 7
   3.5 Training Strategy 8
   3.6 Experimental Results 10
      3.6.1 Evaluation Function 10
      3.6.2 Baseline Model Results 11
      3.6.3 Improved Model Results 11
      3.6.4 Comparative Results Summary 11
   3.7 Training Curves Analysis 12
      3.7.1 Baseline Model – Accuracy and Loss 12
      3.7.2 Improved Model – Accuracy and Loss 12
      3.7.3 Comparative Discussion 13
4. Conclusion 13
5. References 14

1. Introduction:

Medical image analysis is an important application of machine learning, especially in the field of healthcare, where automated systems can assist doctors in diagnosis and decision making. One common and serious disease is pneumonia, which affects the lungs and can be life-threatening if it is not detected early. Chest X-ray images are widely used by medical professionals to identify pneumonia, but manual inspection can be time-consuming and depends heavily on expert experience.

In this project, a Convolutional Neural Network (CNN) is used to automatically classify chest X-ray images into two classes: *Normal* and *Pneumonia*. CNNs are a type of deep learning model that are especially suitable for image data, as they can automatically learn important visual features such as edges, shapes, and textures.

The main goal of this project is to design, train, and evaluate CNN models for pneumonia detection, and to compare a simple baseline model with a more advanced CNN architecture. The performance of the models is evaluated using standard classification metrics, including accuracy, precision, recall, and F1-score, as well as confusion matrices. This allows us to analyze not only how accurate the models are, but also how well they detect positive pneumonia cases.

2. Dataset Description:

The dataset used in this project consists of chest X-ray images collected for the purpose of pneumonia detection. The dataset is organized into three separate subsets: a training set, a validation set, and a test set. Each image belongs to one of two classes: *Normal* (healthy lungs) or *Pneumonia* (infected lungs).

The training set is used to train the CNN models, the validation set is used to monitor performance during training and to prevent overfitting, and the test set is used for final evaluation. All images are resized to a fixed size of 224×224 pixels and normalized by scaling the pixel values to the range [0, 1]. This preprocessing step ensures that the input data is consistent and suitable for neural network training.

Data augmentation techniques are applied to the training set, including random rotations, shifts, zooming, and horizontal flipping. These techniques increase the diversity of the training data and help the model generalize better to unseen images. The validation and test sets are not augmented in order to provide a fair and realistic evaluation of the trained models.

3. Methodology and Interpretation:

3.1 Overall Workflow

The overall workflow of the system starts with loading and preprocessing the chest X-ray dataset, followed by training two Convolutional Neural Network (CNN) models: a baseline model and an improved model. The baseline model represents a simple CNN architecture, while the improved model is deeper and includes additional layers for better feature extraction.
The training process uses the training set, while the validation set is used to monitor performance and apply early stopping in order to prevent overfitting. After training, both models are evaluated on an unseen test set. The evaluation is performed using standard classification metrics, including accuracy, precision, recall, F1-score, and confusion matrices. Finally, the results of both models are compared and analyzed.

3.2 Data Loading and Preprocessing

The dataset is loaded using the **ImageDataGenerator** class from the **Keras** library. This class allows efficient loading of images directly from folders and performs preprocessing in real time. The dataset is organized into three folders: training, validation, and test, where each folder contains two subfolders corresponding to the two classes: ***NORMAL*** and ***PNEUMONIA***.
All images are resized to **224×224 pixels** to ensure a consistent input size for the neural networks. Pixel values are normalized by dividing them by 255, which scales them into the range [0, 1]. This normalization improves numerical stability and helps the model converge faster during training.
For the training set, data augmentation is applied, including random rotations, horizontal shifts, zooming, and horizontal flipping. These techniques increase the diversity of the training data and help the model generalize better, reducing the risk of overfitting. The validation and test sets are not augmented, as they are used only for evaluation.
The test set is created with **shuffle=False** to preserve the original order of samples. This is important for correctly computing the confusion matrix and evaluation metrics.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

IMG_SIZE = (224, 224)
BATCH_SIZE = 32

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=10,
    width_shift_range=0.05,
    height_shift_range=0.05,
    zoom_range=0.10,
    horizontal_flip=True
)

test_datagen = ImageDataGenerator(rescale=1./255)
```

```
train_gen = train_datagen.flow_from_directory(
    train_dir,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="binary",
    shuffle=True,
    seed=SEED
)

val_gen = test_datagen.flow_from_directory(
    valid_dir,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="binary",
    shuffle=False
)

test_gen = test_datagen.flow_from_directory(
    test_dir,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="binary",
    shuffle=False
)
```

```
Found 12229 images belonging to 2 classes.
Found 1165 images belonging to 2 classes.
Found 582 images belonging to 2 classes.
Class indices: {'NORMAL': 0, 'PNEUMONIA': 1}
Train samples: 12229 Val samples: 1165 Test samples: 582
```

The dataset contains a total of **12,229 training images**, **1,165 validation images**, and **582 test images**, all belonging to two classes: *NORMAL* and *PNEUMONIA*. The class labels are encoded as NORMAL = 0 and PNEUMONIA = 1. This confirms that the problem is a binary classification task and that the dataset is sufficiently large for training deep learning models.

3.3 Baseline CNN Model

The baseline model is designed as a simple Convolutional Neural Network (CNN) to serve as a reference point for performance comparison. It consists of a single convolutional layer followed by an average pooling layer, a fully connected layer, and a final output layer.
The convolutional layer uses 16 filters with a kernel size of 5×5 and a stride of 5×5, which results in non-overlapping receptive fields. This design satisfies the baseline requirement of the project and allows the model to extract basic visual features from the input images.
An average pooling layer with a pool size of 2×2 is applied after the convolution to reduce the spatial dimensions and the number of parameters. The resulting feature maps are then flattened into a one-dimensional vector and passed to a dense layer with 64 neurons using the ReLU activation function.
Finally, a single neuron with a sigmoid activation function is used as the output layer, since the problem is a binary classification task.

```
baseline_model = Sequential([
    Conv2D(
        filters=16,
        kernel_size=(5, 5),
        strides=(5, 5),
        activation="relu",
        input_shape=(IMG_SIZE[0], IMG_SIZE[1], 3)
    ),
    AveragePooling2D(pool_size=(2, 2)),
    Flatten(),
    Dense(64, activation="relu"),
    Dense(1, activation="sigmoid")
])

baseline_model.compile(
    optimizer=Adam(learning_rate=1e-3),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

baseline_model.summary()
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 44, 44, 16)	1,216
average_pooling2d (AveragePooling2D)	(None, 22, 22, 16)	0
flatten (Flatten)	(None, 7744)	0
dense (Dense)	(None, 64)	495,680
dense_1 (Dense)	(None, 1)	65

Total params: 496,961 (1.90 MB)
Trainable params: 496,961 (1.90 MB)
Non-trainable params: 0 (0.00 B)

The baseline CNN model contains a total of **496,961 trainable parameters**. After the convolution and pooling operations, the feature maps are reduced to a size of **22×22×16**, which are then flattened into a vector of length **7,744**.

Most of the model parameters are located in the first dense layer, which connects the flattened features to 64 neurons. This indicates that the baseline model relies heavily on fully connected layers rather than deep convolutional feature extraction. As a result, the model is relatively simple and mainly captures basic visual patterns in the images.

This baseline architecture provides a useful reference point for evaluating the performance improvements achieved by the deeper CNN model.

3.4 Improved CNN Model

The improved model is a deeper Convolutional Neural Network designed to achieve better performance by learning more complex and hierarchical features from the chest X-ray images. Unlike the baseline model, which uses only one convolutional layer, the improved model consists of three convolutional blocks, each followed by batch normalization and max pooling. The first convolutional layer uses 32 filters, the second uses 64 filters, and the third uses 128 filters. This gradual increase in the number of filters allows the network to capture low-level features such as edges in early layers and more abstract patterns in deeper layers.
Batch normalization is applied after each convolutional layer to stabilize the training process and reduce internal covariate shift. Max pooling layers are used to reduce the spatial dimensions of the feature maps while preserving the most important information.
After the convolutional blocks, a Global Average Pooling layer is used instead of flattening. This reduces the number of parameters and helps prevent overfitting. Two dropout layers are added to randomly deactivate neurons during training, which improves generalization and reduces the risk of overfitting.
Finally, the model includes a dense layer with 64 neurons and a sigmoid output layer for binary classification.

```
improved_model = Sequential([
    Conv2D(32, (3,3), padding="same", activation="relu",
           input_shape=(IMG_SIZE[0], IMG_SIZE[1], 3)),
    BatchNormalization(),
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3), padding="same", activation="relu"),
    BatchNormalization(),
    MaxPooling2D((2,2)),

    Conv2D(128, (3,3), padding="same", activation="relu"),
    BatchNormalization(),
    MaxPooling2D((2,2)),

    GlobalAveragePooling2D(),
    Dropout(0.4),

    Dense(64, activation="relu"),
    Dropout(0.3),
    Dense(1, activation="sigmoid")
])

improved_model.compile(
    optimizer=Adam(learning_rate=1e-3),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

improved_model.summary()
```

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 224, 224, 32)	896
batch_normalization (BatchNormalization)	(None, 224, 224, 32)	128
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_2 (Conv2D)	(None, 112, 112, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 112, 112, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_3 (Conv2D)	(None, 56, 56, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 56, 56, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 128)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0
dropout (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 1)	65

Total params: 102,465 (400.25 KB)
Trainable params: 102,017 (398.50 KB)
Non-trainable params: 448 (1.75 KB)

The improved CNN model contains a total of **102,465 parameters**, which is significantly fewer than the baseline model, despite being deeper. This reduction is mainly due to the use of Global Average Pooling instead of flattening, which avoids creating a very large fully connected layer. The model gradually reduces the spatial dimensions of the input image from **224×224** to **28×28**, while increasing the number of feature channels from **32 to 128**. This design allows the network to extract rich hierarchical features while keeping the number of parameters relatively low.
The inclusion of batch normalization and dropout layers improves training stability and generalization, making the improved model more robust and less prone to overfitting compared to the baseline.

3.5 Training Strategy

Both models are trained using the Adam optimizer and binary cross-entropy loss, which is suitable for binary classification problems. The training process uses the training set, while the validation set is used to monitor the model's performance during training.
Early stopping is applied based on the validation loss. If the validation loss does not improve for several consecutive epochs, training is stopped automatically and the best model weights are restored. This strategy prevents overfitting and avoids unnecessary training time.
In addition, a learning rate reduction strategy is applied. When the validation loss stops improving, the learning rate is reduced by a factor of 0.5. This allows the model to make smaller and more precise updates during later stages of training, which helps improve convergence.
Class weights are also used to handle possible class imbalance between the Normal and Pneumonia classes. This ensures that the model does not become biased toward the majority class.
An epoch is one complete pass of the entire training dataset through the neural network during training.

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau

callbacks = [
    EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True),
    ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2),
    ModelCheckpoint("best_model.keras", monitor="val_loss", save_best_only=True)
]
```

```
EPOCHS = 25

history_baseline = baseline_model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=EPOCHS,
    callbacks=callbacks,
    class_weight=class_weight
)
```

```
history_improved = improved_model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=EPOCHS,
    callbacks=callbacks,
    class_weight=class_weight
)
```

Baseline Training Log

```
Epoch 1/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 164s 419ms/step - accuracy: 0.7068 - loss: 0.5313 - val_accuracy: 0.9253 - val_loss: 0.2106 - learning_rate: 0.0010
Epoch 2/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 145s 379ms/step - accuracy: 0.8525 - loss: 0.3372 - val_accuracy: 0.9330 - val_loss: 0.1784 - learning_rate: 0.0010
Epoch 3/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 143s 373ms/step - accuracy: 0.8711 - loss: 0.3074 - val_accuracy: 0.9339 - val_loss: 0.1779 - learning_rate: 0.0010
Epoch 4/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 140s 366ms/step - accuracy: 0.8791 - loss: 0.2831 - val_accuracy: 0.8635 - val_loss: 0.3253 - learning_rate: 0.0010
Epoch 5/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 139s 362ms/step - accuracy: 0.8939 - loss: 0.2555 - val_accuracy: 0.8721 - val_loss: 0.2937 - learning_rate: 0.0010
Epoch 6/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 139s 363ms/step - accuracy: 0.9057 - loss: 0.2330 - val_accuracy: 0.9476 - val_loss: 0.1416 - learning_rate: 5.0000e-04
Epoch 7/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 140s 366ms/step - accuracy: 0.9072 - loss: 0.2243 - val_accuracy: 0.8901 - val_loss: 0.2525 - learning_rate: 5.0000e-04
Epoch 8/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 150s 391ms/step - accuracy: 0.9022 - loss: 0.2271 - val_accuracy: 0.9330 - val_loss: 0.1771 - learning_rate: 5.0000e-04
Epoch 9/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 155s 404ms/step - accuracy: 0.9093 - loss: 0.2218 - val_accuracy: 0.9468 - val_loss: 0.1502 - learning_rate: 2.5000e-04
Epoch 10/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 157s 410ms/step - accuracy: 0.9116 - loss: 0.2182 - val_accuracy: 0.9176 - val_loss: 0.2003 - learning_rate: 2.5000e-04
Epoch 11/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 158s 413ms/step - accuracy: 0.9181 - loss: 0.2096 - val_accuracy: 0.9262 - val_loss: 0.1832 - learning_rate: 1.2500e-04
```

The baseline model shows rapid improvement during the first few epochs. Training accuracy increases from **70.7% in epoch 1** to approximately **91.8% by epoch 11**, indicating that the

model is learning meaningful patterns from the chest X-ray images. At the same time, the training loss steadily decreases, which confirms that the optimization process is effective.
The validation performance is generally strong but shows some fluctuations across epochs. The validation accuracy reaches its highest value of approximately **94.8% at epoch 6**, and the validation loss reaches its minimum value of **0.1416** at the same epoch. After this point, the validation metrics do not improve consistently and occasionally degrade, suggesting that the model reaches its optimal performance around epoch 6.
The learning rate is automatically reduced during training, first from **0.001 to 0.0005**, then to **0.00025**, and finally to **0.000125**, which helps stabilize the learning process and prevents large parameter updates in later epochs. Early stopping is triggered at **epoch 11**, preventing unnecessary further training and ensuring that the model parameters corresponding to the best validation performance are retained for final evaluation.

<u>Improved Model Log</u>

```
Epoch 1/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 184s 457ms/step - accuracy: 0.7753 - loss: 0.4448 - val_accuracy: 0.7391 - val_loss: 1.3414 - learning_rate: 0.0010
Epoch 2/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 167s 436ms/step - accuracy: 0.8334 - loss: 0.3567 - val_accuracy: 0.6635 - val_loss: 0.5901 - learning_rate: 0.0010
Epoch 3/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 168s 440ms/step - accuracy: 0.8568 - loss: 0.3164 - val_accuracy: 0.4240 - val_loss: 1.3190 - learning_rate: 0.0010
Epoch 4/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 169s 442ms/step - accuracy: 0.8845 - loss: 0.2778 - val_accuracy: 0.7373 - val_loss: 0.6546 - learning_rate: 0.0010
Epoch 5/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 168s 439ms/step - accuracy: 0.9020 - loss: 0.2455 - val_accuracy: 0.9116 - val_loss: 0.2168 - learning_rate: 5.0000e-04
Epoch 6/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 167s 435ms/step - accuracy: 0.9097 - loss: 0.2248 - val_accuracy: 0.2627 - val_loss: 15.3808 - learning_rate: 5.0000e-04
Epoch 7/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 167s 435ms/step - accuracy: 0.9194 - loss: 0.2028 - val_accuracy: 0.9030 - val_loss: 0.2019 - learning_rate: 5.0000e-04
Epoch 8/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 169s 442ms/step - accuracy: 0.9229 - loss: 0.1943 - val_accuracy: 0.3828 - val_loss: 4.2831 - learning_rate: 5.0000e-04
Epoch 9/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 165s 430ms/step - accuracy: 0.9218 - loss: 0.2013 - val_accuracy: 0.9356 - val_loss: 0.1577 - learning_rate: 5.0000e-04
Epoch 10/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 170s 444ms/step - accuracy: 0.9292 - loss: 0.1825 - val_accuracy: 0.6275 - val_loss: 1.7685 - learning_rate: 5.0000e-04
Epoch 11/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 167s 437ms/step - accuracy: 0.9290 - loss: 0.1805 - val_accuracy: 0.7408 - val_loss: 3.1597 - learning_rate: 5.0000e-04
Epoch 12/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 167s 436ms/step - accuracy: 0.9335 - loss: 0.1730 - val_accuracy: 0.3820 - val_loss: 3.1670 - learning_rate: 2.5000e-04
Epoch 13/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 171s 445ms/step - accuracy: 0.9285 - loss: 0.1755 - val_accuracy: 0.8936 - val_loss: 0.2537 - learning_rate: 2.5000e-04
Epoch 14/25
383/383 ━━━━━━━━━━━━━━━━━━━━ 169s 440ms/step - accuracy: 0.9416 - loss: 0.1510 - val_accuracy: 0.9262 - val_loss: 0.1957 - learning_rate: 1.2500e-04
```

The improved CNN model shows a clear and steady improvement in training performance across epochs. Training accuracy increases from approximately **77.5% in epoch 1** to about **94.2% by epoch 14**, while the training loss consistently decreases. This indicates that the model is successfully learning increasingly discriminative features from the chest X-ray images as training progresses.
In contrast, the validation accuracy and validation loss exhibit noticeable fluctuations throughout the training process. In several early and intermediate epochs, the validation accuracy drops sharply and the validation loss increases significantly (for example, at epochs 3, 6, 8, and 12). However, the model is also able to recover and achieve strong validation performance in later epochs. The best validation result is observed at **epoch 9**, where the validation accuracy reaches approximately **93.6%** and the validation loss reaches its minimum value of **0.1577**.
This behavior indicates training instability rather than simple overfitting. The deeper architecture of the improved model, combined with data augmentation and a relatively high initial learning rate, can lead to large parameter updates in some training stages, causing temporary degradation in validation performance. The learning rate reduction strategy helps mitigate this issue by gradually decreasing the learning rate, allowing more stable and precise updates in later epochs.

Early stopping plays an important role by automatically terminating training once no consistent improvement in validation loss is observed. As a result, the model parameters corresponding to the best validation performance are retained, despite the fluctuations seen during training. Overall, although the validation metrics do not improve monotonically, the improved model demonstrates effective learning, high training accuracy, and strong peak validation performance. The final evaluation of the model is therefore based on test-set results, which provide an unbiased and reliable measure of its generalization ability.

3.6 Experimental Results

3.6.1 Evaluation Function

After training, both models are evaluated using a unified evaluation function to ensure a fair and consistent comparison. The evaluation is performed on the unseen test set, which was not used during training or validation. This provides an unbiased estimate of the models' generalization performance.

The evaluation function computes prediction probabilities for all test samples and converts them into binary class labels using a fixed threshold of 0.5. Several standard classification metrics are

```python
from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score, precision_recall_fscore_support

def evaluate_binary(model, generator, name, threshold=0.5):
    y_prob = model.predict(generator).ravel()
    y_pred = (y_prob >= threshold).astype(int)
    y_true = generator.classes

    acc = accuracy_score(y_true, y_pred)
    precision, recall, f1, _ = precision_recall_fscore_support(y_true, y_pred, average="binary")
    cm = confusion_matrix(y_true, y_pred)

    print(f"\n===== {name} =====")
    print("Accuracy :", acc)
    print("Precision:", precision)
    print("Recall   :", recall)
    print("F1-score :", f1)
    print("\nConfusion Matrix:\n", cm)
```

```python
    print("\nClassification Report:\n",
          classification_report(
              y_true, y_pred,
              target_names=list(generator.class_indices.keys))
          )
    )

    return {
        "Model": name,
        "Accuracy": acc,
        "Precision": precision,
        "Recall": recall,
        "F1": f1,
        "ConfusionMatrix": cm
    }
```

then calculated, including accuracy, precision, recall, and F1-score. In addition, a confusion matrix and a detailed classification report are generated to provide deeper insight into the models' performance on each class.

Using a single shared evaluation function guarantees that both the baseline and improved models are assessed under identical conditions.

The function first generates prediction probabilities using the trained model. These probabilities are converted into binary predictions using a threshold of 0.5, which is standard for binary classification. Accuracy measures overall correctness, while precision, recall, and F1-score focus on the model's performance in detecting pneumonia cases. The confusion matrix provides a detailed breakdown of true positives, true negatives, false positives, and false negatives.

3.6.2 Baseline Model Results

```
===== Baseline =====
Accuracy : 0.9278350515463918
Precision: 0.9624060150375939
Recall   : 0.9343065693430657
F1-score : 0.9481481481481482

Confusion Matrix:
 [[156  15]
 [ 27 384]]
```

The baseline model achieves an accuracy of **92.78%** on the test set. The high precision value (**96.24%**) indicates that when the model predicts pneumonia, it is usually correct. The recall value (**93.43%**) shows that the model successfully detects most pneumonia cases, although some are still missed.
The confusion matrix shows **27 false negatives**, meaning that 27 pneumonia cases were incorrectly classified as normal. In a medical context, false negatives are critical because they represent missed diagnoses. While the baseline model performs well overall, reducing false negatives remains an important objective.

3.6.3 Improved Model Results

```
===== Improved =====
Accuracy : 0.9398625429553265
Precision: 0.9372093023255814
Recall   : 0.9805352798053528
F1-score : 0.9583828775267539

Confusion Matrix:
 [[144  27]
 [  8 403]]
```

The improved CNN model achieves a higher test accuracy of **93.99%**, outperforming the baseline model. Most notably, the recall increases to **98.05%**, indicating that the improved model detects almost all pneumonia cases. This results in only **8 false negatives**, a substantial reduction compared to the baseline model.
Although the precision of the improved model (**93.72%**) is slightly lower than that of the baseline, the significant improvement in recall leads to a higher F1-score (**95.84%**). This indicates a better balance between precision and recall, which is especially important in medical diagnosis tasks.

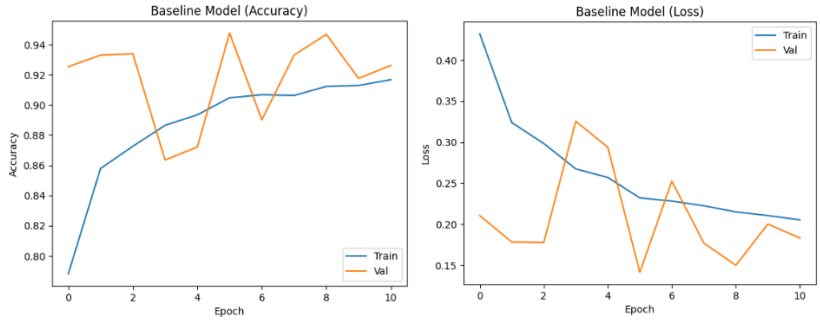
3.6.4 Comparative Results Summary

	Model	Accuracy	Precision	Recall	F1
0	Baseline	0.927835	0.962406	0.934307	0.948148
1	Improved	0.939863	0.937209	0.980535	0.958383

The improved CNN model outperforms the baseline model in terms of overall accuracy and F1-score. Most importantly, it significantly improves recall and reduces the number of false negatives. This demonstrates that a deeper architecture with batch normalization, dropout, and global average pooling can achieve better generalization while using fewer parameters than the baseline model.
From a medical perspective, the improved model is more suitable for pneumonia detection because it minimizes missed positive cases, even at the cost of a small increase in false positives.

3.7 Training Curves Analysis

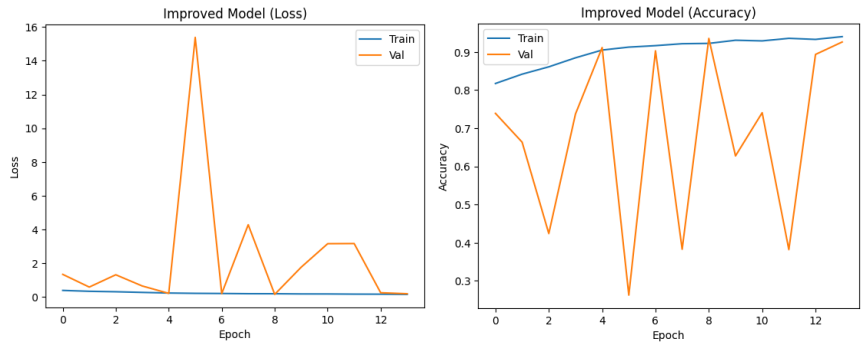
3.7.1 Baseline Model – Accuracy and Loss



The baseline model shows a smooth and stable learning behavior. Training accuracy increases steadily across epochs, indicating consistent learning from the training data. Validation accuracy closely follows the training accuracy and remains relatively high, with only minor fluctuations. The loss curves further confirm this behavior. Training loss decreases monotonically, while validation loss shows a general downward trend with small oscillations. The lowest validation loss occurs around epoch 6, which aligns with the point where the model achieves its best validation accuracy. After this point, validation performance does not improve consistently, justifying the use of early stopping.

Overall, the baseline model demonstrates stable convergence with limited overfitting. The close alignment between training and validation curves suggests that the model generalizes reasonably well, although its capacity is limited due to its simple architecture.

3.7.2 Improved Model – Accuracy and Loss



The improved CNN model exhibits a different training behavior compared to the baseline model. Training accuracy increases steadily and reaches high values, indicating that the deeper architecture is able to learn complex and discriminative features from the training data. However, the validation accuracy and validation loss show significant fluctuations across epochs. In several epochs, validation loss increases sharply and validation accuracy drops, followed by recovery in later epochs. This behavior is also visible in the loss curve, where validation loss occasionally spikes to very high values before decreasing again.

These fluctuations indicate **training instability rather than classical overfitting**. The deeper architecture, combined with data augmentation and a relatively high initial learning rate, can cause large parameter updates that temporarily degrade validation performance. The learning rate reduction mechanism helps mitigate this issue by stabilizing training in later epochs.

Despite the unstable validation curves, the improved model achieves strong peak validation performance and, more importantly, superior test-set results. Early stopping ensures that the model checkpoint with the best validation loss is selected for final evaluation.

3.7.3 Comparative Discussion

Comparing both models, the baseline model shows more stable and predictable training behavior, while the improved model demonstrates higher learning capacity but increased sensitivity during training. This trade-off is expected when moving from a shallow to a deeper CNN architecture.

Although the improved model's validation curves are noisier, its final test performance is superior, particularly in terms of recall and F1-score. This indicates that the improved model generalizes better to unseen data, even if its validation behavior during training is less smooth. In medical image classification tasks, final test performance—especially recall—is more important than perfectly smooth training curves. Therefore, the improved model is considered more effective for pneumonia detection.

4. Conclusion

In this project, Convolutional Neural Networks were applied to the task of pneumonia detection using chest X-ray images. Two models were designed and evaluated: a simple baseline CNN and a deeper improved CNN architecture. The goal was to compare their performance and analyze the effect of architectural improvements on classification accuracy and generalization.

The baseline model demonstrated stable training behavior and achieved good performance on the test set, with an accuracy of **92.78%** and an F1-score of **94.81%**. However, it relied heavily on fully connected layers and showed limitations in reducing false negative cases, which is an important concern in medical diagnosis tasks.

The improved CNN model achieved better overall performance, reaching a test accuracy of **93.99%** and an F1-score of **95.84%**. Most importantly, it significantly improved recall, correctly identifying **98.05%** of pneumonia cases and reducing the number of false negatives from **27** in the baseline model to **8**. Although the improved model showed fluctuations in validation performance during training, early stopping and learning rate reduction ensured that the best-performing model was selected.

Overall, the results demonstrate that a deeper CNN architecture with batch normalization, dropout, and global average pooling can outperform a simpler baseline model while using fewer parameters. From a medical perspective, the improved model is more suitable for pneumonia detection, as it prioritizes minimizing missed positive cases. Future work could explore more stable training strategies, alternative data augmentation techniques, or transfer learning approaches to further improve performance.

5. References

[1] Chest X-Ray Pneumonia Dataset, Kaggle.
[2] Course Slides, *Convolutional Neural Networks (CNNs)*, Machine Learning Course, Birzeit University, Fall 2025.
[3] *Simple explanation of convolutional neural network*, YouTube video by codebasics (2020).
https://www.youtube.com/watch?v=zfiSAzpy9NM
[4]*Convolutional Neural Networks - Deep Learning basics with Python, TensorFlow and Keras p.3*, YouTube video by 3Blue1Brown (2019).
https://www.youtube.com/watch?v=WvoLTXIjBYU